

JavaScript

Complete Revision Notes

Basic → Intermediate → Advanced
Synchronous + Asynchronous Programming
Definitions • Explanations • Roman Urdu • Practical Examples

Revision-first approach

Designed for learners who already know JavaScript and want a strong, structured revision resource. Difficult ideas get a little more explanation and a larger example, while routine syntax stays short.

Language style

English is used for technical definitions and syntax. Roman Urdu is added where it makes the concept easier to remember naturally.

Core mental model

JavaScript code is executed by a runtime. Normal code runs synchronously on the call stack. Asynchronous operations can finish later, after which the runtime schedules callbacks or Promise reactions. This is the foundation for understanding fetch, async/await, timers, and the event loop.

```
Synchronous:  
code → call stack → execute  
  
Asynchronous:  
start operation → host/runtime handles it  
→ result ready → callback queued  
→ event loop → callback runs
```

Table of Contents

1. JavaScript Foundations
2. Operators and Control Flow
3. Functions
4. Strings, Numbers and Template Literals
5. Arrays and Functional Array Methods
6. Objects, Destructuring and References
7. Scope, Hoisting and Closures
8. this, Classes and Prototypes
9. Modules
10. DOM and Browser APIs
11. Errors and Debugging
12. JSON and Regular Expressions
13. Synchronous Programming
14. Asynchronous Programming
15. Event Loop and JavaScript Execution
16. Advanced JavaScript
17. Real-World JavaScript Patterns

1. JavaScript Foundations

What is JavaScript?

Definition

JavaScript is a high-level, dynamically typed programming language used to build interactive web applications and also many non-browser applications.

Explanation

Think of JavaScript as the language that describes behavior and logic. In a frontend project, HTML gives structure, CSS controls presentation, and JavaScript reacts to user actions, processes data, communicates with APIs, and updates the UI. JavaScript itself is the language; the browser or another runtime provides extra APIs such as the DOM, timers, and fetch.

Roman Urdu

Simple lafzon mein: JavaScript page ko interactive banata hai. Button click, form validation, calculations, API se data lana, aur UI update karna - yeh sab JS se hota hai.

Short example

```
const message = "Hello JavaScript";
console.log(message);
```

Revision tip

Remember the difference between the language and the runtime environment.

JavaScript Runtime

Definition

A runtime is the environment that executes JavaScript and provides extra APIs around the language.

Explanation

A browser includes a JavaScript engine plus browser APIs. The engine executes JS; browser APIs handle things such as the DOM, timers, and network requests. Node.js is another runtime with its own APIs.

Roman Urdu

Browser mein JS sirf language nahi hoti; browser humein extra tools bhi deta hai, jaise document, fetch aur setTimeout.

Short example

```
console.log(typeof document); // object in a browser
console.log(typeof setTimeout); // function
```

Variables: const, let, var

Definition

Variables are bindings used to refer to values. const and let are block-scoped; var is function-scoped.

Explanation

Use const when the binding should not be reassigned, and let when reassignment is required. const does not make an object immutable; it means the variable binding itself cannot be reassigned.

Roman Urdu

Normal rule yaad rakho: pehle const socho, agar value ko dobara assign karna ho to let use karo. var modern code mein usually avoid kiya jata hai.

Short example

```
const name = "Salman";
let count = 0;
count = count + 1;
```

Data Types

Definition

JavaScript has primitive types such as string, number, bigint, boolean, undefined, null, and symbol; objects are non-primitive values.

Explanation

Primitives represent individual immutable values. Objects are collections and are accessed through references. Arrays and functions are also objects in JavaScript's type system.

Roman Urdu

Primitive ko simple value samjho. Object ko data ka box samjho jisme multiple properties ho sakti hain.

Short example

```
const name = "Ali";
const age = 20;
const active = true;
const user = { name, age };
```

typeof

Definition

The typeof operator returns a string describing the type category of its operand.

Explanation

It is useful for quick checks, but remember quirks such as typeof null being 'object' and arrays also returning 'object'.

Roman Urdu

typeof basic checking ke liye useful hai, lekin har case mein exact type batane ke liye enough nahi hota.

Short example

```
console.log(typeof "JS");    // "string"
console.log(typeof 42);      // "number"
console.log(typeof null);    // "object"
console.log(typeof []);      // "object"
```

Type Conversion

Definition

Type conversion means explicitly changing a value from one data type to another.

Explanation

Explicit conversion makes code easier to understand. Common tools are `Number()`, `String()`, `Boolean()`, `parseInt()`, and `parseFloat()`.

Roman Urdu

Agar tum khud type change karte ho to usay explicit conversion keh sakte ho. Yeh implicit coercion se zyada predictable hota hai.

Short example

```
const age = Number("20");
const text = String(20);
const active = Boolean(1);
```

Type Coercion

Definition

Type coercion is JavaScript automatically converting values during an operation.

Explanation

It is most noticeable with operators such as `+`, `-`, `==`, and in boolean conditions. Learn the rules so unexpected outputs do not surprise you, but use explicit conversion when clarity is important.

Roman Urdu

JavaScript kabhi kabhi khud type convert kar deta hai. Is behavior ko coercion kehte hain.

Short example

```
console.log("5" + 2); // "52"
console.log("5" - 2); // 3
```

Truthy and Falsy

Definition

Truthy values behave like true in boolean contexts; falsy values behave like false.

Explanation

Common falsy values are `false`, `0`, `-0`, `0n`, `,`, `null`, `undefined`, and `NaN`. Almost everything else is truthy, including `[]` and `{}`.

Roman Urdu

Condition mein agar value ko true/false context mein use karo to JS usay truthy ya falsy treat karta hai. Empty array aur empty object bhi truthy hote hain.

Short example

```
console.log(Boolean(0)); // false
console.log(Boolean("JS")); // true
console.log(Boolean([])); // true
```

2. Operators and Control Flow

Arithmetic Operators

Definition

Arithmetic operators perform mathematical operations.

Explanation

The most common operators are +, -, *, /, %, and **. The remainder operator is especially useful for even/odd checks and cycles.

Roman Urdu

Basic maths operators hain. % remainder deta hai aur ** power ke liye use hota hai.

Short example

```
console.log(10 % 3); // 1
console.log(2 ** 3); // 8
```

Comparison Operators

Definition

Comparison operators compare values and return a boolean.

Explanation

Prefer strict equality (===) and strict inequality (!==). They compare without the same type coercion used by loose equality.

Roman Urdu

Revision ke liye simple rule: mostly === aur !== use karo, taake type coercion ki wajah se surprise na aaye.

Short example

```
console.log(5 === 5); // true
console.log("5" === 5); // false
```

Logical AND &&

Definition

The && operator returns the first falsy operand, or the last operand when all operands are truthy.

Explanation

This means && is not limited to returning true or false. It is commonly used for guards and conditional rendering.

Roman Urdu

Agar left side falsy ho to wahi return hoti hai; warna right side evaluate hoti hai.

Short example

```
console.log(true && "yes");
console.log(false && "yes");
console.log("Salman" && "some string");
```

Logical OR ||

Definition

The || operator returns the first truthy operand, or the last operand when none are truthy.

Explanation

It is often used for fallbacks, but remember that 0, false, and an empty string are falsy, so || may replace values that are actually valid.

Roman Urdu

OR fallback dene ke liye useful hai, lekin 0 ko bhi falsy samjhega. Isi wajah se kabhi ?? better hota hai.

Short example

```
console.log("name" || "Guest");  
console.log("" || "Guest");  
console.log(0 || 10); // 10
```

Nullish Coalescing ??

Definition

The ?? operator uses the right-hand value only when the left-hand value is null or undefined.

Explanation

This is useful when 0, false, or an empty string should remain valid data.

Roman Urdu

Agar tum sirf null aur undefined par fallback chahte ho to ?? use karo. 0 ko yeh preserve karta hai.

Short example

```
const count = 0;  
console.log(count ?? 10); // 0  
console.log(null ?? 10); // 10
```

Optional Chaining ?.

Definition

Optional chaining safely accesses a property or calls a method when an intermediate value may be null or undefined.

Explanation

Instead of manually checking every level, ?. stops the chain safely and returns undefined if the needed value is unavailable.

Roman Urdu

Nested object mein property missing ho sakti ho to ?. bohat useful hai. Error ke bajaye undefined milta hai.

Short example

```
const user = {};  
console.log(user.address?.city); // undefined
```

if / else

Definition

if/else conditionally executes different blocks of statements.

Explanation

Use it when your program needs to choose between multiple actions. More branches can be handled with else if.

Roman Urdu

Condition true ho to pehla block, warna doosra block chalta hai.

Short example

```
const age = 20;
if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

Ternary Operator

Definition

The ternary operator is a compact expression that chooses one of two values.

Explanation

It is useful for small decisions where a single value is needed. Do not turn complex business logic into deeply nested ternaries.

Roman Urdu

Ternary asal mein if/else ka short expression hai jab tumhein direct value chahiye.

Short example

```
const age = 20;
const result = age >= 18 ? "Adult" : "Minor";
```

switch

Definition

switch selects a block based on a matching expression value.

Explanation

Use break to prevent accidental fall-through. switch can be useful when many fixed cases are being compared.

Roman Urdu

Jab ek value ke multiple fixed cases hon, switch readable ho sakta hai.

Short example

```
const day = "Mon";
switch (day) {
  case "Mon":
    console.log("Start");
    break;
  default:
    console.log("Other");
}
```

Loops

Definition

Loops repeat code while a condition or sequence requires it.

Explanation

Know for, while, do...while, for...of, and for...in. for...of is generally used for iterable values such as arrays; for...in is mainly used for object keys.

Roman Urdu

Loop ka matlab same logic ko multiple times run karna. Array ke values ke liye for...of ya array methods aksar cleaner hote hain.

Short example

```
for (let i = 0; i < 3; i++) {
  console.log(i);
}

for (const value of [10, 20]) {
  console.log(value);
}
```

3. Functions

Function

Definition

A function is a reusable block of code that can receive inputs and return an output.

Explanation

Functions let you separate logic into small reusable units. A function may take zero or many parameters and may return any JavaScript value.

Roman Urdu

Function ko ek reusable machine samjho: input do, logic chalega, aur result return ho sakta hai.

Short example

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(2, 3));
```

Parameters and Arguments

Definition

Parameters are names in the function definition; arguments are the actual values passed during a call.

Explanation

Default parameters provide fallback values when an argument is undefined.

Roman Urdu

Parameter function ke andar naam hota hai; argument call karte waqt actual value hoti hai.

Short example

```
function greet(name = "Guest") {  
  return `Hello ${name}`;  
}  
  
console.log(greet("Salman"));  
console.log(greet());
```

return

Definition

return ends the current function execution and sends a value back to the caller.

Explanation

Once return executes, later statements in that function do not run. A function with no explicit return produces undefined.

Roman Urdu

return function se bahar result bhejta hai aur function ko wahi rok deta hai.

Short example

```
function square(x) {  
  return x * x;  
  console.log("Never runs");  
}  
console.log(square(5));
```

Arrow Functions

Definition

Arrow functions provide shorter function syntax and inherit this from their surrounding scope.

Explanation

They are especially common as callbacks. A concise-body arrow function implicitly returns its expression.

Roman Urdu

Arrow function short syntax deta hai aur is ka apna this nahi hota.

Short example

```
const add = (a, b) => a + b;  
const doubled = [1, 2, 3].map(x => x * 2);
```

Callback Functions

Definition

A callback is a function passed to another function so it can be called later or as part of an operation.

Explanation

Callbacks appear in array methods, event handlers, timers, and older asynchronous APIs.

Roman Urdu

Ek function ko doosre function ko argument ke taur par dena callback pattern hai.

Short example

```
function process(value, callback) {  
  const result = value * 2;  
  callback(result);  
}  
  
process(5, value => console.log(value));
```

Higher-Order Functions

Definition

A higher-order function takes a function as an argument, returns a function, or both.

Explanation

This idea is central to functional JavaScript. Array methods such as map, filter, and reduce are classic examples.

Roman Urdu

Aisa function jo function ko receive ya return kare usay higher-order function kehte hain.

Short example

```
function apply(fn, value) {  
  return fn(value);  
}  
  
console.log(apply(x => x * 2, 5));
```

Rest Parameters

Definition

Rest parameters collect remaining function arguments into an array.

Explanation

Rest is useful when a function should accept a flexible number of arguments.

Roman Urdu

Yahan ... remaining arguments ko collect karta hai. Isay rest kehte hain.

Short example

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b, 0);  
}  
  
console.log(sum(1, 2, 3, 4));
```

4. Strings, Numbers and Template Literals

Strings

Definition

A string represents text.

Explanation

Strings are immutable, so methods such as `toUpperCase` or `slice` return new strings instead of changing the original string.

Roman Urdu

String text hota hai. String methods original string ko usually mutate nahi karte; new string return karte hain.

Short example

```
const text = "JavaScript";
console.log(text.length);
console.log(text.toUpperCase());
```

Template Literals

Definition

Template literals are strings written with backticks that can embed JavaScript expressions using `${}`.

Explanation

They make dynamic text easier to read and also support multiline strings.

Roman Urdu

Backticks wali strings dynamic text ke liye bohat useful hain. `${}` ke andar JS expression likh sakte ho.

Short example

```
const name = "Salman";
const pages = 120;
const summary = `${name} has ${pages} pages.`;
console.log(summary);
```

Important String Methods

Definition

String methods provide common operations such as searching, slicing, splitting, and replacing text.

Explanation

Useful methods to revise: `includes`, `startsWith`, `endsWith`, `slice`, `substring`, `split`, `replace`, `replaceAll`, `trim`, `toLowerCase`, and `toUpperCase`.

Roman Urdu

Revision ke liye string methods ko categories mein yaad rakho: check, cut, split, replace, clean, case change.

Short example

```
const text = " hello world ";
console.log(text.trim());
console.log(text.includes("world"));
console.log(text.split(" "));
```

Number and Math

Definition

Number is JavaScript's main numeric type, while Math provides common mathematical operations.

Explanation

JavaScript uses floating-point numbers for ordinary Number values, which can cause precision details such as $0.1 + 0.2$ not being exactly 0.3.

Roman Urdu

Normal numbers ke liye Number use hota hai. Decimal calculations mein floating-point precision ka concept yaad rakho.

Short example

```
console.log(Math.round(4.6));  
console.log(Math.floor(4.9));  
console.log(0.1 + 0.2);
```

BigInt

Definition

BigInt represents integers larger than the safe integer range of Number.

Explanation

BigInt literals use `n`. BigInt and Number should not be directly mixed in arithmetic without explicit conversion.

Roman Urdu

Bohat bade integers ke liye BigInt use hota hai. Iske end par `n` lagta hai.

Short example

```
const big = 9007199254740993n;  
console.log(big);
```

5. Arrays and Functional Array Methods

Array

Definition

An array is an ordered collection of values indexed from zero.

Explanation

Arrays are objects and can contain values of different types, although consistent data structures are usually easier to maintain.

Roman Urdu

Array ko ordered list samjho. Index 0 se start hota hai.

Short example

```
const fruits = ["apple", "banana", "mango"];
console.log(fruits[0]);
console.log(fruits.length);
```

Array Destructuring

Definition

Array destructuring extracts values from array positions into variables.

Explanation

It is useful when you already know which positions you need.

Roman Urdu

Position ke hisaab se values variables mein nikal sakte ho.

Short example

```
const genres = ["adventure", "history", "fantasy"];
const [primary, secondary, ...others] = genres;
console.log(primary, secondary, others);
```

Spread Operator

Definition

Spread expands iterable values into another array or object literal.

Explanation

Spread is commonly used for shallow copies, combining arrays, and creating updated objects.

Roman Urdu

Spread ka kaam opposite hai rest se: rest collect karta hai, spread expand karta hai.

Short example

```
const genres = ["adventure", "history"];
const newGenres = [...genres, "fantasy"];
console.log(newGenres);
```

map

Definition

map creates a new array by transforming each element.

Explanation

The callback returns the new value for each item. The original array is not mutated by map.

Roman Urdu

Har item ko transform karke naya array milta hai.

Short example

```
const pages = [100, 200, 300];
const doubled = pages.map(page => page * 2);
console.log(doubled);
```

filter

Definition

filter creates a new array containing only elements that pass a condition.

Explanation

Use it when your question is basically: which items should remain?

Roman Urdu

Jo items condition pass karein sirf woh new array mein aate hain.

Short example

```
const books = [
  { title: "A", pages: 300 },
  { title: "B", pages: 700 }
];

const longBooks = books.filter(book => book.pages > 500);
```

find

Definition

find returns the first element that satisfies a condition, or undefined if no element matches.

Explanation

Use find when you need one item rather than an array of matching items.

Roman Urdu

Agar ek matching object chahiye to find, multiple chahiye to filter.

Short example

```
const books = [
  { id: 1, title: "A" },
  { id: 2, title: "B" }
];

const book = books.find(book => book.id === 2);
```

reduce

Definition

reduce processes an array and combines its values into one accumulated result.

Explanation

You provide an accumulator and a callback. reduce can calculate totals, build objects, group data, and much more, but use it only when the one-result model actually fits the problem.

Roman Urdu

reduce ko accumulator wali machine samjho: har item previous result ke saath combine hota jata hai.

Short example

```
const pages = [100, 200, 300];
const total = pages.reduce((acc, page) => acc + page, 0);
console.log(total);
```

some and every

Definition

some checks whether at least one element passes; every checks whether all elements pass.

Explanation

These methods are useful when you need a boolean answer rather than the matching items themselves.

Roman Urdu

Question yes/no wala ho to some ya every useful ho sakta hai.

Short example

```
const scores = [80, 65, 90];
console.log(scores.some(score => score > 85));
console.log(scores.every(score => score >= 50));
```

sort

Definition

sort orders an array in place and mutates the original array.

Explanation

For numbers, pass a comparator. If you need the original array unchanged, copy it first or use toSorted() where available.

Roman Urdu

sort original array ko change kar sakta hai. Is point ko zaroor yaad rakho.

Short example

```
const numbers = [10, 2, 5];
const sorted = numbers.slice().sort((a, b) => a - b);
console.log(sorted);
console.log(numbers);
```

6. Objects, Destructuring and References

Object

Definition

An object is a collection of named properties.

Explanation

Objects are commonly used to model entities and structured data. Properties can hold primitives, arrays, objects, and functions.

Roman Urdu

Object mein key-value data hota hai. Real-world entity ko represent karne ke liye bohat common hai.

Short example

```
const user = {  
  name: "Salman",  
  age: 20,  
  active: true  
};  
  
console.log(user.name);
```

Object Destructuring

Definition

Object destructuring extracts properties into variables.

Explanation

It makes code shorter when you repeatedly access the same properties.

Roman Urdu

Agar object se same properties baar baar nikalni hon to destructuring clean lagti hai.

Short example

```
const book = {  
  title: "Book A",  
  author: "Ali",  
  pages: 400  
};  
  
const { title, author, pages } = book;
```

Object Spread

Definition

Object spread copies enumerable own properties into a new object.

Explanation

It is useful for shallow copies and immutable updates. Nested objects still share references.

Roman Urdu

Spread se object ka shallow copy banta hai. Nested data ho to deep copy automatically nahi hoti.

Short example

```
const user = { name: "Ali", age: 20 };
const updated = { ...user, age: 21 };
console.log(updated);
```

Reference Behavior

Definition

Objects are values accessed through references, so two variables can point to the same object.

Explanation

Changing a shared object through one reference can be visible through another reference. This is a key reason mutation can cause bugs.

Roman Urdu

Object assignment ko simple copy mat samjho; aksar reference share hota hai.

Short example

```
const a = { score: 10 };
const b = a;
b.score = 20;
console.log(a.score); // 20
```

Object.keys, values and entries

Definition

These static methods return arrays containing keys, values, or key-value pairs.

Explanation

They are useful when you want to loop over object data with array methods.

Roman Urdu

Object ko array jaisi processing mein lana ho to keys/values/entries useful hain.

Short example

```
const user = { name: "Ali", age: 20 };
console.log(Object.keys(user));
console.log(Object.values(user));
console.log(Object.entries(user));
```

7. Scope, Hoisting and Closures

Scope

Definition

Scope determines where a variable can be accessed.

Explanation

JavaScript commonly involves global, module, function, and block scopes. `let` and `const` are block-scoped; `var` is function-scoped.

Roman Urdu

Scope basically batata hai variable kahan available hai aur kahan nahi.

Short example

```
function test() {  
  const inside = 10;  
  console.log(inside);  
}  
  
test();  
// inside is not available here
```

Lexical Scope

Definition

Lexical scope means variable access depends on where a function is written in the source code.

Explanation

A nested function can access variables from its surrounding scopes because those scopes are lexically related.

Roman Urdu

Function jahan likha gaya hai, us jagah ke surrounding variables ko access kar sakta hai.

Short example

```
const outerValue = 10;  
  
function show() {  
  console.log(outerValue);  
}  
  
show();
```

Hoisting

Definition

Hoisting describes how declarations are processed before execution of their surrounding scope.

Explanation

Function declarations can be called before their declaration. `let` and `const` are not safely accessible before initialization even though their bindings are created during scope setup.

Roman Urdu

Hoisting ko yeh samjho ke JS execution se pehle declarations ke structure ko process karta hai. Iska matlab yeh nahi ke har variable literally top par move ho jata hai.

Short example

```
sayHi();

function sayHi() {
  console.log("Hi");
}
```

Temporal Dead Zone

Definition

The temporal dead zone is the period from entering a scope until a `let` or `const` binding is initialized.

Explanation

Accessing the variable inside this period causes a `ReferenceError`.

Roman Urdu

`let` aur `const` declaration se pehle access karne par error aata hai. Isi period ko TDZ kehte hain.

Short example

```
console.log(x); // ReferenceError
let x = 10;
```

Closure

Definition

A closure happens when a function keeps access to variables from its lexical scope even after the outer function has finished.

Explanation

Closures are extremely important. They appear in callbacks, event handlers, factories, data privacy patterns, and many framework concepts. The inner function remembers the environment it was created in.

Roman Urdu

Closure ko aise samjho: outer function khatam ho gaya, lekin inner function ko outer ke variables ki yaad rehti hai.

Short example

```
function counter() {  
  let count = 0;  
  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const next = counter();  
console.log(next()); // 1  
console.log(next()); // 2
```

8. this, Classes and Prototypes

this

Definition

this is a special value whose meaning depends mainly on how a function is called.

Explanation

In a normal object method call, this usually refers to the object before the dot. Arrow functions do not create their own this; they inherit it from the surrounding scope.

Roman Urdu

this ka meaning sirf function dekh kar decide nahi hota; function ko kis tarah call kiya gaya hai yeh important hai.

Short example

```
const user = {  
  name: "Ali",  
  show() {  
    console.log(this.name);  
  }  
};  
  
user.show(); // Ali
```

call, apply and bind

Definition

These methods let you control a function's this value; call and apply invoke immediately, while bind returns a new function.

Explanation

call takes arguments separately, apply takes an array-like argument collection, and bind creates a reusable function with the chosen this.

Roman Urdu

Simple yaad rakho: call/apply abhi chalate hain, bind future ke liye function ready karta hai.

Short example

```
function greet(city) {  
  console.log(this.name, city);  
}  
  
const user = { name: "Ali" };  
greet.call(user, "Kohat");
```

Class

Definition

A class is syntax for defining a constructor and shared methods for objects.

Explanation

Classes are built on JavaScript's prototype system. They make object-oriented code easier to read but do not replace the underlying prototype model.

Roman Urdu

Class ko object banane ka blueprint samajh sakte ho.

Short example

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
  
  greet() {  
    return `Hello ${this.name}`;  
  }  
}  
  
const user = new User("Salman");  
console.log(user.greet());
```

Inheritance

Definition

Inheritance allows one class to extend another class and reuse its behavior.

Explanation

The child class can add new methods and override inherited behavior.

Roman Urdu

Child class parent ki properties aur methods reuse kar sakti hai.

Short example

```
class Admin extends User {
  deleteUser() {
    return "deleted";
  }
}

const admin = new Admin("Salman");
console.log(admin.greet());
```

Prototype

Definition

A prototype is an object from which another object can inherit properties and methods.

Explanation

JavaScript uses prototype-based inheritance. When a property is not found directly on an object, lookup continues through its prototype chain.

Roman Urdu

Prototype ko backup object samjho jahan JS missing property/method dhoondhne ja sakti hai.

Short example

```
const animal = {
  speak() {
    return "sound";
  }
};

const dog = Object.create(animal);
console.log(dog.speak());
```

Prototype Chain

Definition

The prototype chain is the sequence of linked objects searched during property lookup.

Explanation

If an object does not have a property, JavaScript checks its prototype, then that prototype's prototype, and so on until the chain ends.

Roman Urdu

Ek property na mile to JS next inherited level par jati hai. Isi chain ko prototype chain kehte hain.

Short example

```
const base = { x: 10 };
const child = Object.create(base);
console.log(child.x); // 10
```

9. Modules

ES Modules

Definition

ES Modules split JavaScript into files with explicit import and export relationships.

Explanation

Modules make larger projects easier to maintain because dependencies become visible and scoped instead of living in one huge global space.

Roman Urdu

Large project mein code ko files mein divide karke import/export karna modules ka main purpose hai.

Short example

```
export const add = (a, b) => a + b;

// another file
import { add } from "../math.js";
```

Named and Default Exports

Definition

Named exports are imported by their exported name; a default export represents the module's default exported value.

Explanation

Use named exports when a module has several related utilities. A default export is often used for a module's primary value.

Roman Urdu

Named mein exact exported name import karna hota hai; default mein name tum khud rakh sakte ho.

Short example

```
export const x = 1;
export const y = 2;
export default function main() {}

// import { x, y } from "./file.js";
```

10. DOM and Browser APIs

DOM

Definition

The Document Object Model represents an HTML document as a tree of objects.

Explanation

Browser JavaScript can query elements, change text, classes and attributes, create elements, and respond to events through the DOM.

Roman Urdu

DOM browser ke HTML ko JavaScript-accessible object structure mein represent karta hai.

Short example

```
const heading = document.querySelector("h1");
heading.textContent = "Updated Heading";
```

Events

Definition

An event represents something that happens, such as a click, input, submit, keydown, or mouse movement.

Explanation

`addEventListener` connects an event type to a callback. The browser calls that callback when the event occurs.

Roman Urdu

Event basically browser ko batata hai ke kuch hua hai. Tum listener se uska response define karte ho.

Short example

```
const button = document.querySelector("button");

button.addEventListener("click", () => {
  console.log("Clicked");
});
```

Event Object

Definition

The event object contains information about the event being handled.

Explanation

Properties such as `target`, `currentTarget`, `key`, `clientX`, and `preventDefault` can be useful depending on the event.

Roman Urdu

Event ke saath browser tumhein extra information deta hai, jo event object mein milti hai.

Short example

```
button.addEventListener("click", event => {
  console.log(event.target);
});
```

Event Bubbling and Delegation

Definition

Event bubbling means an event usually moves from the target upward through its ancestors.

Explanation

Because of bubbling, you can attach one listener to a parent and inspect `event.target` instead of adding separate listeners to many similar children.

Roman Urdu

Parent ek listener se multiple child buttons handle kar sakta hai. Is pattern ko event delegation kehte hain.

Short example

```
const list = document.querySelector("ul");

list.addEventListener("click", event => {
  if (event.target.matches("button")) {
    console.log("Button:", event.target.textContent);
  }
});
```

localStorage

Definition

localStorage stores string key-value data in the browser and keeps it across page reloads.

Explanation

For objects and arrays, use `JSON.stringify` before storing and `JSON.parse` after reading.

Roman Urdu

Browser mein simple persistent data rakhna ho to localStorage useful hai. Yaad rakho: values strings hoti hain.

Short example

```
const user = { name: "Salman", age: 20 };
localStorage.setItem("user", JSON.stringify(user));

const saved = JSON.parse(localStorage.getItem("user"));
console.log(saved.name);
```

11. Errors and Debugging

Common Error Types

Definition

JavaScript can produce `SyntaxError`, `ReferenceError`, `TypeError`, `RangeError`, and other errors.

Explanation

Learn to read the first line of the error, the message, and the stack trace. The error type often points toward the category of mistake.

Roman Urdu

Error dekh kar sirf line mat dekho; error type aur message ko read karo. Yeh debugging ko fast karta hai.

Short example

```
try {
  JSON.parse("bad json");
} catch (error) {
  console.log(error.name);
  console.log(error.message);
}
```

try, catch, finally

Definition

try contains code that may throw; catch handles the error; finally runs after the outcome.

Explanation

finally is useful for cleanup logic that should happen whether the operation succeeds or fails.

Roman Urdu

try risky kaam karta hai, catch error handle karta hai, aur finally cleanup ke liye useful hai.

Short example

```
try {
  throw new Error("Something failed");
} catch (error) {
  console.log(error.message);
} finally {
  console.log("Finished");
}
```

throw

Definition

throw manually creates an exception and interrupts the current control flow until a catch handles it.

Explanation

Throw meaningful Error objects so callers receive useful messages and stack traces.

Roman Urdu

Jab tum khud kehna chaho ke operation invalid hai, throw use kar sakte ho.

Short example

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }
  return a / b;
}
```

Debugging

Definition

Debugging is the process of finding the cause of incorrect program behavior and fixing it.

Explanation

Use console logging for quick inspection, breakpoints for step-by-step execution, and the Network tab for API problems. Avoid logging sensitive information.

Roman Urdu

Debugging ka best tareeqa guess karna nahi, values aur execution ko observe karna hai.

Short example

```
function add(a, b) {  
  debugger;  
  return a + b;  
}  
  
console.log(add(2, 3));
```

12. JSON and Regular Expressions

JSON

Definition

JSON is a text format used to exchange structured data between systems.

Explanation

JSON.stringify converts a JavaScript value to JSON text. JSON.parse converts valid JSON text back into JavaScript data.

Roman Urdu

API data ke saath JSON bohat common hai. Stringify JS se JSON, parse JSON se JS.

Short example

```
const text = JSON.stringify({ name: "Ali", age: 20 });  
const data = JSON.parse(text);  
console.log(data.name);
```

Regular Expressions

Definition

A regular expression describes a text pattern used for searching, matching, or replacing.

Explanation

Regex is useful for validation and text processing, but complex patterns should be documented because they can become hard to maintain.

Roman Urdu

Regex text ke pattern ko match karne ke liye hota hai. Simple patterns se start karo.

Short example

```
const pattern = /js/i;
console.log(pattern.test("JavaScript")); // true

const value = "abc123";
console.log(/\d+/.test(value)); // true
```

13. Synchronous Programming

Synchronous Programming

Definition

Synchronous code executes in order, and the next statement waits for the current one to finish.

Explanation

Normal variable operations, calculations, and ordinary function calls are synchronous. This is the basic execution model before asynchronous work is introduced.

Roman Urdu

Ek kaam complete hoga, phir next kaam chalega. Ye synchronous flow hai.

Short example

```
console.log("A");
console.log("B");
console.log("C");
// A, B, C
```

Blocking

Definition

Blocking work keeps the current JavaScript execution busy and prevents other JavaScript tasks from running on that thread.

Explanation

A long CPU-heavy loop can freeze a browser interface. Asynchronous APIs help with waiting for external operations, but CPU-heavy work can still require worker threads for true parallelism.

Roman Urdu

Agar JS thread bohat der tak busy rahe to UI respond nahi karegi. Isay blocking behavior samjho.

Short example

```
console.log("Start");
for (let i = 0; i < 1e8; i++) {}
console.log("End");
```

14. Asynchronous Programming

Asynchronous Programming

Definition

Asynchronous programming lets an operation finish later while JavaScript continues executing other work.

Explanation

This is important for timers, HTTP requests, file operations, and other tasks that depend on external or future results. Async does not mean multiple lines of JavaScript literally execute at the same instant on one thread.

Roman Urdu

Simple wording: kaam start karo, result baad mein aaye, tab tak JS doosra useful kaam kar sakti hai.

Short example

```
console.log("Start");

setTimeout(() => {
  console.log("Later");
}, 1000);

console.log("End");
// Start, End, Later
```

setTimeout

Definition

setTimeout schedules a callback after a minimum delay.

Explanation

The callback is not guaranteed to execute at exactly the requested time. If the call stack is busy, the callback waits until JavaScript can run it.

Roman Urdu

1000ms ka matlab exact 1 second par function zaroor chalega nahi; minimum delay samjho.

Short example

```
setTimeout(() => {  
  console.log("Timer callback");  
}, 1000);  
  
console.log("This can run first");
```

Promise

Definition

A Promise represents the eventual result of an asynchronous operation.

Explanation

A Promise has three conceptual states: pending, fulfilled, and rejected. Once it settles, it does not switch to the other settled state.

Roman Urdu

Promise future result ka container samjho. Abhi pending, baad mein success ya failure.

Short example

```
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => resolve("Done"), 500);  
});  
  
promise.then(value => console.log(value));
```

then, catch, finally

Definition

then handles fulfillment, catch handles rejection, and finally runs after the Promise settles.

Explanation

These methods return new Promises, which is why chains can be composed. A rejection can move down the chain until a catch handles it.

Roman Urdu

then success, catch error, finally cleanup. Ye teen async revision ke important pieces hain.

Short example

```
fetch("/data.json")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error))
  .finally(() => console.log("Finished"));
```

async / await

Definition

An async function always returns a Promise; await pauses that async function until the awaited Promise settles.

Explanation

await does not block the whole JavaScript runtime. It pauses only the current async function and gives the runtime a chance to handle other work.

Roman Urdu

Ye bohat important point hai: await poori JS ko freeze nahi karta; sirf current async function ka flow pause hota hai.

Short example

```
async function getUsers() {
  const response = await fetch("/api/users");
  const data = await response.json();
  return data;
}

getUsers().then(users => console.log(users));
```

fetch

Definition

fetch performs an HTTP request and returns a Promise that fulfills with a Response object.

Explanation

A response with HTTP 404 or 500 does not automatically reject fetch. Check response.ok when you want HTTP status failures to become thrown errors.

Roman Urdu

fetch request ka Promise deta hai. Important: network failure aur HTTP error same cheez nahi hain.

Short example

```
async function load() {
  const response = await fetch("/api/users");

  if (!response.ok) {
    throw new Error("HTTP request failed");
  }

  return response.json();
}
```

Promise.all

Definition

Promise.all waits for all input Promises to fulfill and rejects when any one rejects.

Explanation

Use it when tasks are independent and can start together. It reduces total waiting time compared with sequential awaits when requests do not depend on each other.

Roman Urdu

Do independent requests hon to unko parallel start karna better ho sakta hai.

Short example

```
const [users, products] = await Promise.all([
  getUsers(),
  getProducts()
]);

console.log(users, products);
```

Promise.allSettled

Definition

Promise.allSettled waits for all Promises and reports the outcome of every one.

Explanation

It is useful when you care about both successful and failed operations rather than stopping at the first failure.

Roman Urdu

Ek fail ho jaye lekin baaki results bhi chahiye hon to allSettled useful hai.

Short example

```
const results = await Promise.allSettled([
  getUsers(),
  getProducts(),
  getOrders()
]);

console.log(results);
```

Promise.race

Definition

Promise.race settles as soon as the first input Promise settles.

Explanation

It can be used when the first completed result matters, including simple timeout patterns when paired with another Promise.

Roman Urdu

Jo Promise sabse pehle settle ho uska result race se milta hai.

Short example

```
const result = await Promise.race([
  getFromServerA(),
  getFromServerB()
]);
```

Promise.any

Definition

Promise.any fulfills as soon as the first input Promise fulfills and rejects only when all inputs reject.

Explanation

It is useful when several alternative sources exist and any successful answer is acceptable.

Roman Urdu

Agar multiple sources mein se koi ek successful result chahiye ho to any useful hai.

Short example

```
const result = await Promise.any([
  serverA(),
  serverB(),
  serverC()
]);
```

Callback Hell

Definition

Callback hell describes deeply nested callbacks that make asynchronous logic difficult to read and maintain.

Explanation

Promises and async/await provide clearer ways to represent sequential asynchronous dependencies. The real problem is not callbacks themselves, but excessive nesting and difficult control flow.

Roman Urdu

Nested callback ke andar callback aur phir callback ho jaye to code samajhna mushkil ho jata hai.

Short example

```
getUser(id, user => {
  getOrders(user, orders => {
    getTotal(orders, total => {
      console.log(total);
    });
  });
});
```

15. Event Loop and JavaScript Execution

Call Stack

Definition

The call stack tracks the functions currently executing in JavaScript.

Explanation

When a function is called, a stack frame is added. When it returns, that frame is removed. Long synchronous work keeps the stack busy.

Roman Urdu

Call stack ko current JS execution ki stack samjho. Function call hota hai to stack par aata hai.

Short example

```
function a() {
  b();
}

function b() {
  console.log("Inside b");
}

a();
```

Event Loop

Definition

The event loop coordinates when queued asynchronous callbacks can move to the call stack.

Explanation

A simplified mental model is: run current synchronous code; when the stack is empty, process queued microtasks; then continue to eligible tasks. Browser and runtime scheduling details are more nuanced, but this model is excellent for revision.

Roman Urdu

Event loop ka main job yeh decide karna samjho ke queued callback ko kab JS stack par bhejna hai.

Short example

```
console.log("1");

setTimeout(() => console.log("2"), 0);

console.log("3");
// 1, 3, 2
```

Microtask Queue

Definition

A microtask queue holds jobs such as Promise reactions and queueMicrotask callbacks.

Explanation

After the current JavaScript stack becomes empty, microtasks are generally drained before the event loop takes the next task. This is why Promise callbacks often appear before setTimeout callbacks.

Roman Urdu

Promise wala callback aam tor par timer se pehle run hota hai kyun ke microtasks pehle drain hoti hain.

Short example

```
console.log("A");

Promise.resolve().then(() => console.log("B"));

setTimeout(() => console.log("C"), 0);

console.log("D");
// A, D, B, C
```

Task / Macrotask Queue

Definition

Tasks are queued work items such as many timer callbacks and browser event callbacks.

Explanation

Once synchronous code and relevant microtasks are handled, the runtime can move on to another task. Different host environments can have more detailed scheduling rules.

Roman Urdu

Timer aur bohat se browser events task queue mein aa sakte hain.

Short example

```
setTimeout(() => console.log("Timer task"), 0);
```

Execution Order

Definition

A useful revision order is: synchronous code → microtasks → next task, while the exact event-loop model depends on the host environment.

Explanation

When you see a Promise and setTimeout in the same example, first write the synchronous output, then Promise callbacks, then timer callbacks. This mental process solves many interview and exam questions.

Roman Urdu

Output predict karte waqt pehle normal code likho, phir Promise callbacks, phir timers/events.

Short example

```
console.log("Start");
Promise.resolve().then(() => console.log("Promise"));
setTimeout(() => console.log("Timer"), 0);
console.log("End");
// Start, End, Promise, Timer
```

16. Advanced JavaScript

Immutability

Definition

Immutability means avoiding direct changes to an existing value and instead creating updated data.

Explanation

Primitive values are immutable by nature. For objects and arrays, immutability is a coding style that helps make state changes predictable and easier to track.

Roman Urdu

Purane object ko directly mutate karne ke bajaye updated copy banana immutability ka common pattern hai.

Short example

```
const user = { name: "Ali", age: 20 };
const updatedUser = { ...user, age: 21 };
console.log(user.age); // 20
console.log(updatedUser.age); // 21
```

Pure Function

Definition

A pure function returns the same result for the same inputs and has no observable side effects.

Explanation

Pure functions are easier to test, reason about, and reuse. A function that logs, changes external variables, updates the DOM, or performs a network request has side effects.

Roman Urdu

Same input, same output, aur bahar koi unwanted change nahi - yeh pure function ka core idea hai.

Short example

```
const add = (a, b) => a + b;

console.log(add(2, 3));
console.log(add(2, 3));
```

IIFE

Definition

An IIFE is an Immediately Invoked Function Expression that runs as soon as it is created.

Explanation

It was historically used to create a private scope before ES modules became common. You may still see it in older code.

Roman Urdu

Function ko define karte hi turant run karna ho to IIFE pattern hota hai.

Short example

```
(function () {
  const secret = 42;
  console.log(secret);
})();
```

Iterator

Definition

An iterator is an object with a `next()` method that returns an object with value and done.

Explanation

Iterables such as arrays provide an iterator through `Symbol.iterator`, enabling `for...of`.

Roman Urdu

Iterator ko step-by-step values dene wali machine samjho. Har `next()` par next value milti hai.

Short example

```
const iterator = [10, 20][Symbol.iterator]();

console.log(iterator.next());
console.log(iterator.next());
console.log(iterator.next());
```

Generator

Definition

A generator function can pause and resume execution using `yield`.

Explanation

Calling a generator returns an iterator. Generators are useful when values should be produced lazily rather than all at once.

Roman Urdu

Generator `yield` par ruk sakta hai aur baad mein `next()` se continue hota hai.

Short example

```
function* ids() {
  yield 1;
  yield 2;
  yield 3;
}

const gen = ids();
console.log(gen.next());
console.log(gen.next());
```

Map

Definition

Map is a key-value collection whose keys can be values of any type.

Explanation

Unlike plain objects, Map has explicit map operations such as set, get, has, and delete, and preserves insertion order during iteration.

Roman Urdu

Jab dedicated key-value collection chahiye ho aur keys strings tak limited na hon to Map useful hai.

Short example

```
const map = new Map();
map.set("id", 101);
map.set("name", "Salman");

console.log(map.get("name"));
```

Set

Definition

Set is a collection that stores unique values.

Explanation

It is useful for removing duplicates and checking membership. Sets preserve insertion order for iteration.

Roman Urdu

Duplicate values remove karni hon to Set bohat handy hai.

Short example

```
const unique = [...new Set([1, 1, 2, 3, 3])];
console.log(unique);
```

WeakMap

Definition

WeakMap stores object keys in a way that does not keep those objects alive solely because of the WeakMap reference.

Explanation

It is mainly an advanced memory/metadata tool. WeakMap keys must be objects.

Roman Urdu

Advanced use case hai: object ke saath hidden metadata attach karna without unnecessarily preventing garbage collection.

Short example

```
const metadata = new WeakMap();
const user = {};
metadata.set(user, { loaded: true });
console.log(metadata.get(user));
```

Symbol

Definition

Symbol is a unique primitive value often used as a property key.

Explanation

Each Symbol is unique, even if it has the same description. Symbols are also used by built-in language protocols.

Roman Urdu

Symbol unique key banane ke liye useful ho sakta hai taake accidental property-name collision na ho.

Short example

```
const id = Symbol("id");
const user = { [id]: 123 };
console.log(user[id]);
```

Proxy

Definition

Proxy creates a wrapper that can intercept fundamental operations on another object.

Explanation

You can intercept get, set, has, deleteProperty, and more. This is powerful metaprogramming and is also used by frameworks.

Roman Urdu

Proxy se tum object ke normal operations ke beech custom behavior insert kar sakte ho.

Short example

```
const user = new Proxy({}, {
  get(target, prop) {
    return prop in target ? target[prop] : "Not found";
  }
});

console.log(user.name);
```

Reflect

Definition

Reflect provides function-based methods for common object operations.

Explanation

Reflect is especially useful inside Proxy handlers because it can forward operations while preserving normal semantics.

Roman Urdu

Reflect ko object operations ka standard helper set samjho, especially Proxy ke saath.

Short example

```
const user = { name: "Ali" };
console.log(Reflect.get(user, "name"));
```

Garbage Collection

Definition

Garbage collection automatically reclaims memory that is no longer reachable by the program.

Explanation

You normally do not manually free JavaScript memory. Memory leaks happen when unnecessary references remain reachable, such as forgotten event listeners, long-lived caches, or accidental global references.

Roman Urdu

JS memory automatically clean karta hai, lekin agar tum unnecessary references zinda rakho to memory leak ho sakta hai.

Short example

```
let data = { huge: new Array(10000).fill("x") };  
data = null; // removes this reference
```

Debounce

Definition

Debouncing delays a function until calls stop happening for a specified period.

Explanation

It is useful for search inputs or resize events, where you do not want to perform expensive work on every keystroke or pixel movement.

Roman Urdu

User typing karta rahe to function har key par run na ho; rukne ke baad run ho - debounce.

Short example

```
function debounce(fn, delay) {  
  let timer;  
  
  return (...args) => {  
    clearTimeout(timer);  
    timer = setTimeout(() => fn(...args), delay);  
  };  
}  
  
const search = debounce(value => console.log(value), 400);
```

Throttle

Definition

Throttling limits how often a function can run during a time interval.

Explanation

It is useful for scroll, mousemove, and resize events when you want regular updates but not one execution for every event.

Roman Urdu

Events bohat fast aa rahe hon aur tum function ko limited rate par chalana chaho to throttle useful hai.

Short example

```
function throttle(fn, delay) {
  let ready = true;

  return (...args) => {
    if (!ready) return;
    ready = false;
    fn(...args);
    setTimeout(() => ready = true, delay);
  };
}
```

17. Real-World JavaScript Patterns

API Request Flow

Definition

A common frontend data flow is: user action → function → request → response → parse data → update state/UI.

Explanation

This one flow combines events, functions, Promises, async/await, JSON, error handling, and rendering. Understanding the complete chain is more important than memorizing isolated APIs.

Roman Urdu

Real project mein concepts alag alag nahi rehte; API call karte waqt multiple JS concepts ek saath use hote hain.

Short example

```
async function loadUsers() {
  try {
    const response = await fetch("/api/users");
    if (!response.ok) throw new Error("Request failed");

    const users = await response.json();
    console.log(users);
  } catch (error) {
    console.error(error);
  }
}
```

Sequential vs Parallel Async

Definition

Sequential async operations wait for one result before starting the next; parallel operations start independent tasks together.

Explanation

If task B depends on A, sequential waiting is necessary. If A and B are independent, Promise.all can reduce total waiting time.

Roman Urdu

Dependency ho to sequential, dependency na ho to parallel. Yeh performance aur readability dono ke liye important decision hai.

Short example

```
const user = await getUser();
const orders = await getOrders(user.id); // depends on user

const [users, products] = await Promise.all([
  getUsers(),
  getProducts()
]);
```

Error Handling in Async Code

Definition

Async failures should be handled with try/catch around awaited operations or catch() on Promise chains.

Explanation

Do not ignore rejected Promises. Decide what the application should show when a request fails: retry, fallback data, error message, or another flow.

Roman Urdu

Async error ko ignore mat karo. User ko meaningful fallback ya error state dena real app mein important hai.

Short example

```
async function loadData() {
  try {
    const data = await getData();
    return data;
  } catch (error) {
    console.error("Could not load data:", error);
    return [];
  }
}
```

Final Quick Revision

Basics

Remember

Variables, data types, coercion, truthy/falsy, operators, conditions, loops

Functions

Remember

Parameters, return, arrow functions, callbacks, higher-order functions, rest

Data

Remember

Arrays, objects, destructuring, spread, map, filter, find, reduce, sort

Scope

Remember

Lexical scope, hoisting, TDZ, closures, this

Objects

Remember

Classes, inheritance, prototypes, prototype chain, Map, Set, Symbol

Browser

Remember

DOM, events, bubbling, delegation, localStorage

Async

Remember

Promise, then/catch, async/await, fetch, all, allSettled, race, any

Event Loop

Remember

Call stack, microtasks, tasks, execution order

Advanced

Remember

Immutability, pure functions, generators, iterators, Proxy, Reflect

Performance

Remember

Debounce, throttle, avoiding unnecessary work, async parallelism

Best revision method

Do not only reread. For each difficult concept, close the notes and explain it in your own words, then write a small example from memory. For asynchronous JavaScript, manually predict the output before running the code.